AutoBill — Requisitos e Critérios de Aceite para Implementação

Monitor inteligente de faturas e alertas de vencimento (Python, Telegram, Docker)

Versão: 1.0 | **Uso:** pessoal ou familiar (single-tenant) | **Canal principal:** Telegram | **Execução:** Docker Compose

---

1. Visão Geral

1.1 Objetivo

O AutoBill recebe faturas e boletos (PDF ou imagem), extrai vencimento, valor, beneficiário e linha digitável, **valida os dados matematicamente**, guarda tudo em banco local e envia alertas proativos antes e no dia do vencimento, com a linha digitável pronta para copiar e botões para marcar como pago.

1.2 Escopo

Dentro do escopo:

- Entrada por pasta monitorada, por envio direto ao bot do Telegram e, opcionalmente, por caixa de e-mail (IMAP).
- Extração por LLM com visão, com alternativa local (OCR) e validação por regras.
- Confirmação humana quando a confiança é baixa.
- Alertas escalonados (antecipados, no dia e em atraso) e comandos interativos.
- Controle de status e histórico, exportação e backup.

Fora do escopo (v1):

- Pagamento automático ou integração bancária (Open Finance, PIX).
- Acesso a contas bancárias ou a portais de concessionárias.
- Aplicativo móvel ou painel web.
- Multiusuário com cadastro público.

1.3 Personas

Persona	Necessidade
Morador / chefe de família	Não esquecer vencimentos e copiar o código sem erro

---

Persona	Necessidade
Casal ou família (grupo no Telegram)	Contas compartilhadas e marcação de quem pagou
Autonomo / MEI	Controle simples de contas recorrentes
Mantenedor	Sistema pequeno, testado e fácil de hospedar

---

2. Requisitos Funcionais (RF)

2.1 Ingestão

RF01 — Monitorar pasta de entrada.

- Dado um arquivo PDF, PNG, JPG ou WEBP colocado em `data/inbox/`, Quando o monitor detecta o arquivo (em até 60 s), Então ele entra na fila de processamento.
- Dado um arquivo ainda sendo copiado (tamanho instável), Então ele só é processado após estabilizar por 5 s.
- Dado um formato não suportado ou arquivo acima de 20 MB, Então ele é movido para `data/rejeitados/` e o usuário é avisado com o motivo.
- Dado o mesmo conteúdo enviado duas vezes (mesmo hash SHA-256), Então a segunda cópia é ignorada e o usuário é informado.

RF02 — Receber arquivos pelo Telegram. O usuário encaminha foto ou PDF ao bot.

- Dado um documento ou foto enviado por chat autorizado, Quando o bot o recebe, Então responde "Recebido, processando..." em até 2 s.
- Dado um chat não autorizado, Então o arquivo é descartado sem processamento.
- Dado um álbum de várias imagens, Então cada imagem é tratada como fatura distinta, a menos que o usuário use `/paginas` para agrupar.

RF03 — Monitorar e-mail (opcional). Ler anexos de uma caixa dedicada via IMAP.

- Dado `IMAP_ENABLED=true`, Quando chega e-mail não lido com anexo PDF ou imagem, Então os anexos são baixados, processados e o e-mail é marcado como lido ou movido para uma pasta "AutoBill".
- Dado e-mail sem anexo, Então o corpo é ignorado e nada é gravado.
- Dado remetente fora da lista `ALLOWED_SENDERS` (quando configurada), Então o e-mail é ignorado.

RF04 — Lançamento manual. `/adicionar` abre um assistente passo a passo (beneficiário, valor, vencimento, linha digitável opcional).

- Dado dados válidos, Então a fatura é criada com origem `manual`.

- Dado data inválida ou valor negativo, Então o assistente informa o erro e repete a pergunta.

2.2 Extração e validação

RF05 — Extração de dados. Extrair beneficiário, valor, vencimento e linha digitável (ou código de barras).

- Dado uma fatura legível de concessionária, Quando processada, Então os quatro campos são extraídos em até 30 s.
- Dado um PDF com várias páginas, Então todas as páginas (limite de 5) são analisadas, e não apenas a primeira.
- Dado PDF protegido por senha, Então o bot pergunta a senha (apagada da memória após o uso) ou informa que não foi possível abrir.
- Dado campo não encontrado, Então ele fica nulo e nunca é preenchido por suposição.

RF06 — Extração por múltiplas estratégias. Ordem: (1) texto embutido do PDF e regex; (2) leitura do código de barras na imagem; (3) LLM com visão; (4) OCR local como alternativa.

- Dado um PDF com camada de texto, Então a linha digitável é extraída por regex antes de qualquer chamada ao LLM.
- Dado `EXTRACTOR=local`, Então nenhum conteúdo do documento sai do ambiente do usuário.
- Dado falha de uma estratégia, Então a próxima é tentada e o log registra qual foi usada.

RF07 — Validação da linha digitável (núcleo de confiabilidade). Aplicar regras da seção 3.

- Dado uma linha com dígito verificador incorreto, Então a fatura não é ativada e o usuário é notificado para revisão.
- Dado uma linha com tamanho diferente de 47 ou 48 dígitos, Então ela é rejeitada.
- Dado a linha válida, Então o código de barras de 44 dígitos correspondente também é calculado e guardado.

RF08 — Verificação cruzada. Comparar o que o LLM leu com o que está codificado na linha.

- Dado boleto bancário, Quando o valor do código difere do valor extraído em mais de R$ 0,01, Então a fatura entra em revisão com ambos os valores exibidos.
- Dado boleto bancário com fator de vencimento, Quando a data calculada difere da extraída, Então a fatura entra em revisão.
- Dado ausência de valor no código (valor zero, "qualquer valor"), Então prevalece o valor extraído e o campo `valor_confirmado` fica falso.

RF09 — Confirmação humana. Faturas com baixa confiança ou divergências exigem confirmação.

- Dado status `EM_REVISAO`, Então o bot envia os dados lidos (e o arquivo original) com botões "Confirmar", "Corrigir" e "Descartar".
- Dado "Corrigir", Então o usuário edita um campo por vez por mensagem e o sistema revalida antes de confirmar.
- Dado fatura em revisão, Então nenhum alerta de vencimento é enviado até a confirmação, mas um lembrete de revisão pendente é enviado em 24 h.
- Dado `AUTO_CONFIRM=true` e todas as validações aprovadas, Então a fatura é ativada automaticamente.

RF10 — Prevenção de duplicidade.

- Dado uma nova fatura com a mesma linha digitável de uma existente, Então ela não é duplicada e o usuário recebe o aviso com link para a original.
- Dado mesma concessionária, mesmo valor e mesmo vencimento sem linha digitável, Então o sistema pergunta se é duplicata.

2.3 Gestão de faturas

RF11 — Ciclo de vida. Estados: `EM_REVISAO`, `PENDENTE`, `PAGO`, `VENCIDA`, `DESCARTADA`.

- Dado a data atual posterior ao vencimento e status `PENDENTE`, Então o estado passa a `VENCIDA` na próxima verificação diária.
- Dado `PAGO` ou `VENCIDA`, Quando o usuário paga uma fatura vencida e a marca, Então ela vai para `PAGO` com a data real informada.

RF12 — Marcar como pago. Por botão inline ou `/pago <id>`.

- Dado o botão "✅ Paguei", Quando tocado, Então o status muda para `PAGO`, a mensagem é editada para mostrar "Pago em dd/mm" e os alertas futuros são cancelados.
- Dado `/pago 7` com ID inexistente, Então o bot responde "Fatura não encontrada".
- Dado a fatura já paga, Então o bot informa e não duplica registros.
- Dado `/desfazer`, Então a última baixa é revertida para `PENDENTE` dentro de 24 h.

RF13 — Soneca e reagendamento. Botão "⏰ Lembrar depois".

- Dado o botão, Então o usuário escolhe 1 h, 3 h ou amanhã e o alerta é reenviado no horário escolhido.

RF14 — Consultas. `/pendentes`, `/proximos [dias]`, `/mes [AAAA-MM]`, `/fatura <id>`, `/pagas`.

- Dado `/pendentes`, Então são listadas as faturas pendentes ordenadas por vencimento, com total em reais.
- Dado `/mes 2026-11`, Então são exibidos total previsto, total pago e total pendente do mês.
- Dado `/fatura 7`, Então o bot mostra todos os campos, o estado e reenvia o arquivo original.

RF15 — Edição e remoção. `/editar <id>` e `/remover <id>`.

- Dado `/remover`, Então o bot pede confirmação e faz exclusão lógica (recuperável por 30 dias).

RF16 — Contas recorrentes (extra). Detectar contas mensais pelo beneficiário.

- Dado 2 faturas consecutivas do mesmo beneficiário com ~30 dias de intervalo, Então o sistema marca a conta como recorrente e prevê o próximo vencimento.
- Dado a previsão e nenhuma fatura recebida até 5 dias antes, Então o bot avisa "A fatura de X ainda não chegou".

2.4 Alertas

RF17 — Alertas escalonados. Configuráveis por usuário. Padrão: 3 dias antes, 1 dia antes, no dia e diário em atraso.

- Dado uma fatura pendente com vencimento em 3 dias, Quando chega 07:30 no fuso do usuário, Então um único alerta é enviado.
- Dado a mesma fatura no dia seguinte (2 dias), Então nenhum alerta é enviado (não há regra para D-2).
- Dado fatura vencida e não paga, Então um alerta de atraso é enviado diariamente por até 7 dias e depois semanalmente.
- Dado faturas futuras fora da janela de alerta, Então nenhum alerta é gerado para elas.

RF18 — Idempotência dos alertas. Cada alerta é registrado.

- Dado reinicialização do serviço às 07:31, Então o alerta das 07:30 não é reenviado.
- Dado serviço fora do ar às 07:30 e retomado às 09:00, Então os alertas do dia são enviados em até 5 min após a retomada (*misfire grace*).

RF19 — Conteúdo do alerta.

- Dado um alerta, Então contém: ícone de urgência, beneficiário, valor em formato brasileiro (R$ 156,88), vencimento (dd/mm/aaaa) com dias restantes, linha digitável em bloco copiável e botões "✅ Paguei", "⏰ Lembrar depois" e "📄 Ver arquivo".
- Dado texto com caracteres especiais (nomes com `_`, `*`, `.`), Então o envio não falha por erro de formatação.

- Dado fatura sem linha digitável, Então o alerta informa "código não disponível" e oferece reenviar o arquivo.

RF20 — Resumos periódicos.

- Dado domingo às 18:00, Então o bot envia o resumo da semana (a vencer, vencidas, pagas).
- Dado o primeiro dia do mês, Então envia o fechamento do mês anterior.

2.5 Configuração e operação

RF21 — Preferências. `/config` ajusta horário do alerta, antecedencias, fuso, moeda e confirmação automática.

- Dado `/config hora 08:15`, Então os próximos alertas usam esse horário.
- Dado `/config antecedencia 5,2,0`, Então os alertas ocorrem em D-5, D-2 e no dia.

RF22 — Controle de acesso. Apenas chats autorizados.

- Dado `chat_id` fora de `ALLOWED_CHAT_IDS`, Então o bot não responde e registra a tentativa.
- Dado grupo autorizado, Então todos os membros do grupo podem marcar contas como pagas e o registro guarda quem o fez.

RF23 — Exportação e backup.

- Dado `/exportar`, Então o bot envia um CSV com todas as faturas (sem o arquivo original).
- Dado `BACKUP_ENABLED=true`, Então uma cópia do banco é gravada diariamente em `data/backup/` com retenção de 14 cópias.

RF24 — Privacidade e exclusão. `/apagar_tudo` e retenção configurável.

- Dado `/apagar_tudo` com confirmação dupla, Então banco, arquivos e backups são removidos.
- Dado `FILE_RETENTION_DAYS=90`, Então arquivos originais de faturas pagas há mais de 90 dias são apagados, mantendo os dados textuais.

RF25 — Saúde e diagnóstico. `/status` e endpoint `/healthz`.

- Dado `/status`, Então o bot informa versão, horário do último processamento, itens na fila, erros recentes e próximo alerta.

---

3. Regras de Negócio de Boletos e Faturas

Estas regras são o diferencial de confiabilidade do projeto. Confirme os detalhes na documentação vigente da FEBRABAN antes de implementar.

ID	Regra	Critério de aceite
RN01	Linha digitável de boleto bancário tem 47 dígitos; de arrecadação (concessionárias, tributos) tem 48 dígitos e começa com o dígito 8	Entrada com outro tamanho é rejeitada; espaços, pontos e hífens são removidos antes da validação
RN02	Boleto bancário: os três primeiros campos têm dígito verificador pelo módulo 10, e o dígito geral do código de barras segue o módulo 11	Teste com boletos reais (anonimizados) e casos com um dígito alterado: 100% dos erros detectados
RN03	Arrecadação: cada um dos 4 blocos de 12 dígitos tem DV; o módulo (10 ou 11) é definido pelo terceiro dígito do código	Teste com exemplos de ambos os módulos e casos corrompidos
RN04	Conversão entre linha digitável e código de barras (44 dígitos)	Conversão de ida e volta reproduz exatamente a entrada nos casos de teste
RN05	Boleto bancário: valor nos dígitos de valor do código (em centavos) e vencimento pelo fator de vencimento, considerando o reinicio do fator iniciado em 22/02/2025	Datas calculadas conferem com 10 boletos de referência; fatores ambiguos geram revisão
RN06	Arrecadação: o valor pode estar no código e o vencimento pode não estar; o código de moeda e o identificador de valor definem se o valor é efetivo ou referência	Valor só é marcado como confirmado quando extraído do código e coincide com o texto
RN07	Valores monetários são armazenados em centavos (inteiro)	Nenhum cálculo usa ponto flutuante; teste de soma de 1.000 faturas sem erro de arredondamento
RN08	Datas são armazenadas em ISO 8601 (AAAA-MM-DD) e comparadas no fuso configurado	Alerta das 07:30 ocorre no horário local mesmo com servidor em UTC

ID	Regra	Critério de aceite
RN09	Vencimento em sábado, domingo ou feriado: o sistema alerta pelo vencimento nominal e informa se é dia não útil	Mensagem exibe "vence em sábado; pague na sexta" quando aplicável
RN10	Fatura sem vencimento identificado não gera alertas automáticos e fica em revisão	Teste com fatura sem data: status `EM_REVISAO`

---

4. Requisitos Não Funcionais (RNF)

ID	Categoria	Requisito	Critério de aceite mensurável
RNF01	Acurácia	Extração correta de dados críticos	Em conjunto de 50 faturas reais anonimizadas: linha digitável correta ≥ 98%, valor ≥ 98%, vencimento ≥ 97% (após validação)
RNF02	Segurança da informação	Linha digitável nunca incorreta chega ao usuário como válida	0 alertas com linha reprovada na validação
RNF03	Desempenho	Processamento de uma fatura	Mediana ≤ 15 s com LLM em nuvem; ≤ 45 s com OCR local
RNF04	Confiabilidade	Alertas não são perdidos nem duplicados	Teste com reinicializações forçadas: 0 perdas e 0 duplicatas em 100 ciclos
RNF05	Persistência	Dados sobrevivem a reinicializações	Banco em volume Docker; modo WAL do SQLite ativo
RNF06	Privacidade (LGPD)	Dados financeiros e pessoais minimizados	Uso do provedor em nuvem exige consentimento explícito em configuração; modo local disponível
RNF07	Segurança	Segredos fora do código e dos logs	gitleaks limpo; logs mascaram linha digitável (exibem apenas os 6 últimos dígitos)

ID	Categoria	Requisito	Critério de aceite mensurável
RNF08	Disponibilidade	Serviço reinicia sozinho	`restart: unless-stopped` e healthcheck; recupera em < 1 min
RNF09	Observabilidade	Logs estruturados	Log JSON com id da fatura, etapa, estratégia e tempo
RNF10	Resiliência externa	Retry com backoff para LLM e Telegram	429/5xx tratados com até 3 tentativas; falha final vai para fila de reprocessamento
RNF11	Custo	Baixo custo operacional	Funciona somente com recursos gratuitos ou locais; custo mensal estimado no `/status`
RNF12	Manutenibilidade	Código tipado e testado	Cobertura ≥ 85% em validação, banco e alertas; `ruff` e `mypy` sem erros
RNF13	Portabilidade	Linux, macOS, Windows (via Docker)	`docker compose up` funcional em máquina limpa em < 5 min
RNF14	Usabilidade	Mensagens claras e em português (pt-BR)	Todos os textos externalizados; datas e valores em formato brasileiro
RNF15	Acessibilidade	Leitura fácil no celular	Alerta com até 12 linhas; informação essencial não depende de emoji

5. Modelo de Dados

Tabela	Campos principais
`bills`	`id`, `issuer`, `issuer_normalized`, `amount_cents`, `due_date`, `barcode_line`, `barcode_44`, `kind` (bancario, arrecadacao, outro), `status`, `source` (pasta, telegram, email, manual), `file_hash`, `file_path`, `confidence`, `amount_confirmed`, `paid_at`, `paid_by`, `recurring_id`, `created_at`, `deleted_at`
`alerts_sent`	`id`, `bill_id`, `rule` (D-3, D-1, D0, ATRASO), `sent_date`, `telegram_message_id` (chave única: bill, rule, data)
`snoozes`	`id`, `bill_id`, `remind_at`, `done`
`settings`	`chat_id`, `alert_time`, `lead_days`, `timezone`, `auto_confirm`, `locale`
`recurring`	`id`, `issuer_normalized`, `avg_amount_cents`, `typical_day`, `last_seen`
`audit_log`	`id`, `chat_id`, `action`, `bill_id`, `created_at`
`processing_queue`	`id`, `file_hash`, `status`, `attempts`, `last_error`, `created_at`

Restrições: `file_hash` e `barcode_line` únicos (quando presentes); `amount_cents` inteiro não negativo; migrações versionadas.

---

6. Arquitetura



Princípios: um único processo assíncrono (bot + agendador + workers); estado sempre no banco; extração separada da validação; nenhuma regra de negócio dentro dos handlers.

---

7. Metodologias

Prática	Aplicação
Kanban leve	Quadro no GitHub Projects com limite de 3 itens em andamento
TDD no núcleo	Validador de linha digitável, cálculo de datas, regras de alerta e idempotência escritos com testes antes do código

Prática	Aplicação
Testes de propriedade (Hypothesis)	Qualquer linha válida alterada em um dígito é reprovada; conversão linha ↔ barras é reversível
Corpus de teste anonimizado	50 faturas reais com dados sensíveis substituídos, mantidas fora do repositório público (ou sintéticas geradas)
Trunk-based + Conventional Commits	PRs pequenos, changelog e SemVer
12-Factor App	Configuração por ambiente, logs em stdout
ADRs	Decisões: LLM em nuvem vs. local, polling vs. webhook, SQLite vs. outro banco
Feature flags	IMAP, auto-confirmação, extração local, recorrência

Definição de Pronto (DoD): critérios de aceite cobertos por testes; `ruff` e `mypy` limpos; nenhum segredo no código; documentação e `/ajuda` atualizados; PR revisado.

---

8. Tecnologias

Camada	Tecnologia	Justificativa
Linguagem	**Python 3.11+**	Ecossistema de OCR, PDF e bots
Bot	**python-telegram-bot v21+** (ou aiogram 3)	Botões inline, handlers assíncronos e conversas
Agendamento	**APScheduler** com jobstore em SQLite	Cron persistente, fuso horário e *misfire grace*
Banco	**SQLite (WAL)** + **SQLAlchemy 2** + **Alembic**	Simples, portátil e com migrações
Validação	**Pydantic v2**	Schema dos dados extraídos e configuração
PDF	**pypdfium2** (renderização) e **pypdf/pdfplumber** (texto)	Texto direto antes de usar visão
Imagens	**Pillow**	Pré-processamento (rotação, contraste)

Camada	Tecnologia	Justificativa
Código de barras	**pyzbar** (+ libzbar) ou **zxing-cpp**	Ler o código ITF da imagem como segunda fonte de verdade
LLM com visão	**google-genai** (Gemini Flash) ou modelo local via **Ollama** (ex.: modelos de visão como Qwen2.5-VL)	Nuvem para precisão, local para privacidade
OCR local	**Tesseract** (pytesseract) ou **PaddleOCR**	Alternativa sem enviar dados para fora
E-mail	**imap-tools**	Leitura de anexos
Monitor de pasta	**watchdog**	Eventos de arquivo sem *polling* constante
HTTP	**httpx**	Cliente assíncrono
Retry	**tenacity**	Backoff exponencial
Datas e feriados	**zoneinfo**, **holidays** (BR)	Fuso horário e dias não úteis
Logs	**structlog**	JSON com mascaramento
Qualidade	**pytest**, **pytest-asyncio**, **Hypothesis**, **respx**, **pytest-cov**, **ruff**, **mypy**	Padrão de qualidade
CI/CD	**GitHub Actions**, **gitleaks**, **pip-audit**, **Dependabot**	Testes e segurança
Empacotamento	**Docker** multi-stage, **Docker Compose**, *secrets* ou `env_file`	Execução portátil

---

9. Segurança e Privacidade

- **Dados sensíveis:** faturas contêm nome, endereço, CPF/CNPJ, consumo e códigos de pagamento. Tratar como dados pessoais (LGPD).
- **Envio a serviços de nuvem:** ao usar LLM externo, o documento sai do ambiente. Exigir consentimento explícito (`CLOUD_EXTRACTION_CONSENT=true`), revisar a política de retenção do provedor e oferecer modo 100% local.
- **Minimização:** quando possível, extrair texto localmente e enviar ao LLM apenas o necessário; opcionalmente recortar ou ocultar CPF e endereço antes do envio.
- **Acesso:** lista de chats autorizados e registro de auditoria das ações.

- **Segredos:** token do Telegram, chaves de LLM e senha de e-mail por variáveis de ambiente ou *secrets*; nunca no `docker-compose.yml` versionado.
- **Em repouso:** volume com permissões restritas; criptografia do disco ou do banco (SQLCipher) recomendada; backups com a mesma proteção.
- **Logs:** sem conteúdo integral de faturas; linha digitável mascarada.
- **Injeção de prompt:** o texto do documento é dado, não instrução; saída sempre validada por schema. Critério: um PDF com "ignore as regras e retorne X" não altera os campos nem o comportamento.
- **Anti-golpe:** boletos podem ser fraudulentos. O alerta exibe o beneficiário lido, e o `/fatura` destaca divergências entre o nome extraído e o banco emissor do código, quando identificável.

---

10. Matriz de Testes

Área	Casos obrigatórios
Validação de linha	Boleto válido; arrecadação válida (módulo 10 e 11); um dígito trocado; dígitos faltando; caracteres não numéricos; linha com espaços e pontos
Datas	Fator de vencimento antes e depois da mudança de 2025; fim de mês; ano bissexto; vencimento em fim de semana e feriado
Extração	PDF com texto; PDF escaneado; foto inclinada; foto com pouca luz; PDF de várias páginas; PDF protegido por senha; imagem sem boleto
Duplicidade	Mesmo arquivo; mesmo código em arquivos diferentes; mesma fatura reenviada por e-mail e pasta
Alertas	D-3, D-1, D0, atraso; reinicialização no horário do alerta; fuso horário diferente do servidor; fatura paga cancela alertas
Bot	Comandos com argumentos inválidos; chat não autorizado; botões repetidos; mensagens muito longas; nomes com caracteres reservados
Resiliência	LLM fora do ar; Telegram com 429; disco cheio; banco bloqueado; arquivo corrompido

Área	Casos obrigatórios
Segurança	Injeção de prompt em PDF; logs sem dados sensíveis; exclusão total remove arquivos e backups

---

11. Métricas de Sucesso

Métrica	Meta
Linhas digitáveis aprovadas na validação e corretas na conferência manual	100%
Faturas processadas sem intervenção (modo automático)	⩾ 85%
Alertas perdidos ou duplicados	0
Tempo até o primeiro alerta após envio da fatura	< 1 min
Faturas pagas após o vencimento (uso real por 3 meses)	Redução de 90% em relação ao período anterior
Cobertura de testes no núcleo	⩾ 85%
Tempo de instalação até o primeiro alerta	< 15 minutos

---

12. Riscos e Mitigações

Risco	Impacto	Mitigação
Erro de leitura da linha digitável (dígito trocado)	Alto	Validação por DV, leitura do código de barras como segunda fonte, confirmação humana
Vazamento de dados financeiros e pessoais	Alto	Modo local, consentimento, minimização, criptografia, retenção curta
Boleto fraudulento	Alto	Exibir beneficiário e banco, alertar divergências

---

Risco	Impacto	Mitigação
Alerta esquecido por falha do serviço	Alto	Healthcheck, *misfire grace*, idempotência, aviso se o agendador parar
Layouts muito diferentes entre concessionárias	Médio	Estratégias em camadas, corpus de teste variado, revisão humana
Custo ou limite de cota do LLM	Médio	Texto do PDF antes do LLM, modelo local, limite diário
Regra de fator de vencimento mal implementada	Médio	Testes com casos antes e depois de 2025, revisão em caso de dúvida
Spam de alertas	Baixo	Regras explícitas por dia, soneca, resumo semanal

---

13. Roadmap

Marco	Entregas	Duração estimada
M0 — Fundação	Repositório, CI, banco e migrações, configuração, Docker	1 semana
M1 — Núcleo de validação	Validador de linha digitável, conversão, datas, valores em centavos, testes de propriedade	1 semana
M2 — Extração	Texto do PDF, LLM com visão, leitura de código de barras, OCR local, fila	2 semanas
M3 — Bot e revisão	Recebimento pelo Telegram, confirmação e correção, comandos de consulta, `/pago` e botões	1 semana
M4 — Alertas	Regras escalonadas, idempotência, soneca, atraso, resumos, fuso	1 semana
M5 — Operação	IMAP, recorrência, exportação, backup, retenção, `/status`, documentação	1 a 2 semanas

---

14. Backlog Inicial (épicos)

Épico	Histórias principais
E1 — Base	Configuração, banco, migrações, logs, Docker, CI
E2 — Regras de boleto	Módulos 10 e 11; tipos 47 e 48 dígitos; conversão; fator de vencimento; valor
E3 — Ingestão	Pasta, Telegram, e-mail, hash e duplicidade, fila
E4 — Extração	PDF texto, visão, barras, OCR, schema, confiança
E5 — Revisão e gestão	Confirmação, correção, ciclo de vida, edição, remoção
E6 — Alertas	Regras, agendador, idempotência, soneca, atraso, resumos
E7 — Consultas e relatórios	`/pendentes`, `/mes`, `/proximos`, exportação, recorrência
E8 — Segurança e operação	Autorização, mascaramento, retenção, backup, `/status`, `/apagar_tudo`

---

15. Pontos de Atenção no Documento Original

1. **Alertas em excesso.** `get_due_bills()` retorna todas as faturas pendentes com vencimento a partir de hoje (e não só as de até 2 dias, como diz o comentário), gerando alerta diário para todas. Falta a regra de janela e o registro de alertas enviados.
2. **Faturas vencidas não alertam.** A consulta usa `due_date >= hoje`; contas em atraso, as mais críticas, ficam sem aviso.
3. **Comando `/pago` não existe.** O sistema não escuta mensagens (só envia), e a função `mark_as_paid` nunca é chamada. Os botões "Marcar como Pago" citados na stack também não estão implementados; é preciso um bot com *handlers* e `callback_query`.
4. **Sem validação da linha digitável.** O LLM pode errar um dígito, e o usuário pagaria o boleto errado. Validação por dígito verificador e leitura do código de barras são essenciais.
5. **Valor em `float`.** Use centavos inteiros (ou `Decimal`) para evitar erros de arredondamento.

6. **Identificação por nome do arquivo.** `file_name UNIQUE` falha com arquivos de mesmo nome (ex.: `fatura.pdf` todo mês) e não detecta o mesmo conteúdo com nomes diferentes. Use hash do arquivo e a linha digitável.
7. **Data nula.** Se `data_vencimento` vier nula, o registro é salvo sem data e nunca alerta. Deve ir para revisão.
8. **Arquivo só é movido se salvo.** Faturas sem linha digitável ou duplicadas ficam na pasta e são reprocessadas a cada 10 minutos (custo de LLM e logs repetidos). Mova para `rejeitados/` ou `revisao/`.
9. **Entrada por e-mail ausente.** O problema cita e-mail, mas não há integração IMAP.
10. **Somente a primeira página do PDF.** Boletos podem estar em outras páginas; o PDF também pode ter texto extraível sem precisar de visão.
11. **Privacidade.** O documento inteiro é enviado a uma API externa sem aviso; faturas têm dados pessoais.
12. **Formatação Markdown no Telegram.** Nomes com `_` ou `*` podem causar erro 400 no envio e o código não verifica a resposta (a notificação falha silenciosamente).
13. **Fuso horário.** `datetime.now()` e o `schedule` usam o horário do servidor; em Docker normalmente é UTC, e o alerta de 07:30 chegaria em horário errado.
14. **Configuração de ambiente.** Variáveis lidas na importação dos módulos antes de carregar o `.env`; o cliente do Gemini pode ser criado sem chave.
15. **Dependências e Docker.** Revise a versão fixada do SDK do Gemini; `poppler-utils` não é necessário com `pypdfium2`; o campo `version` do Compose é obsoleto e as chaves estão em texto no arquivo.
16. **Destino dos arquivos processados.** `os.rename` para `data/processados_...` mistura arquivos na raiz de `data/` e pode falhar entre volumes; use `shutil.move` e uma pasta dedicada.

---

16. Estrutura de Repositório Sugerida

```
autobill/
├── .github/workflows/ci.yml
├── autobill/
|   ├── __init__.py, main.py, config.py
|   ├── bot/ (handlers.py, keyboards.py, auth.py, messages_ptbr.py)
|   ├── ingest/ (folder.py, telegram_files.py, imap.py, queue.py)
|   ├── extract/ (pdf_text.py, barcode_reader.py, llm_vision.py,
ocr_local.py, pipeline.py)
|   ├── rules/ (boleto.py, arrecadacao.py, modulo.py, due_date.py, money.py)
|   ├── alerts/ (policy.py, scheduler.py, formatter.py)
|   ├── db/ (models.py, repository.py, migrations/)
|   └── ops/ (backup.py, retention.py, health.py)
├── data/ (inbox/, rejeitados/, revisao/, processados/, backup/)
├── tests/ (unit, property, extraction, bot, integration,
fixtures_anonimizadas)
├── docs/ (adr/, regras-de-boleto.md)
├── .env.example, docker-compose.yml, Dockerfile
├── pyproject.toml, README.md, LICENSE, CHANGELOG.md
```

---

17. Variáveis de Ambiente

Variável	Descrição	Exemplo
`TELEGRAM_BOT_TOKEN`	Token do @BotFather	(secreto)
`ALLOWED_CHAT_IDS`	Chats autorizados	`123456789`
`EXTRACTOR`	`cloud`, `local` ou `auto`	`auto`
`CLOUD_EXTRACTION_CONSENT`	Consentimento para enviar documentos à nuvem	`false`
`GEMINI_API_KEY`	Chave do LLM em nuvem	(secreto)
`OLLAMA_HOST`, `LLM_VISION_MODEL`	Modelo local com visão	`http://ollama:11434`
`DEFAULT_TIMEZONE`	Fuso padrão	`America/Fortaleza`
`ALERT_TIME`	Horário dos alertas	`07:30`
`LEAD_DAYS`	Antecedências	`3,1,0`

Variável	Descrição	Exemplo
`AUTO_CONFIRM`	Ativar faturas válidas automaticamente	`false`
`IMAP_ENABLED`, `IMAP_HOST`, `IMAP_USER`, `IMAP_PASSWORD`	E-mail opcional	(secreto)
`FILE_RETENTION_DAYS`	Retenção de arquivos originais	`90`
`DB_PATH`	Caminho do banco	`/app/data/autobill.db`

---

18. Próximos Passos Sugeridos

1. Implementar e testar primeiro o validador de linha digitável (M1), pois ele sustenta a confiabilidade de todo o sistema.
2. Reunir 20 a 50 faturas reais anonimizadas para formar o corpus de teste.
3. Criar o bot com `/pago`, botões e confirmação de revisão antes de investir em recursos extras.
4. Decidir, por ADR, entre extração em nuvem e local, considerando privacidade e custo.